TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PULCHOWK CAMPUS

A

FINAL REPORT

ON

SWEEP: TOXIC COMMENTS CLASSIFICATION

SUBMITTED BY

KASHISH BATAJU (PUL077BCT035)

KRISHALA PRAJAPATI (PUL077BCT038)

MAMATA MAHARJAN (PUL077BCT043)

SUBMITTED TO

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

November, 2024

Acknowledgments

We are deeply grateful to the Samsung Innovation Campus and SIC-AI program co-ordinators at Pulchowk Campus for providing us with the opportunity to plan and execute this project.

The inspiration to address the issue of hate speech on social media was sparked by insightful discussions among ourselves, where we analyzed how the hate spreading on the social platforms easily draws us into the negativity and the harmful impact it can have on individuals. Observing how the spread of hate fosters negativity and affects mental well-being motivated us to take action against this pervasive problem.

We are thankful to **Dr. Suresh Pokharel** and **Er. Bishal Debb Pande**, whose feedback on the feasibility of this project and advice on how to get started was crucial in refining our proposal. We are also grateful to **Dr. Aman Shakya**, our supervisor for guiding us throughout the project and giving us important insights on what more can be performed.

Contents

List of Figures

1. Introduction

1.1 Background

With 5.61 billion unique mobile users at the start of 2024, more than 66 percent of all the people on Earth now use the internet where the active social media user identities are equivalent to 62.3 percent of the world's population. The latest social media user identities figure has increased by 5.6 percent over the past year, with 266 million new users starting to use social media for the first time over the course of 2023. [1]

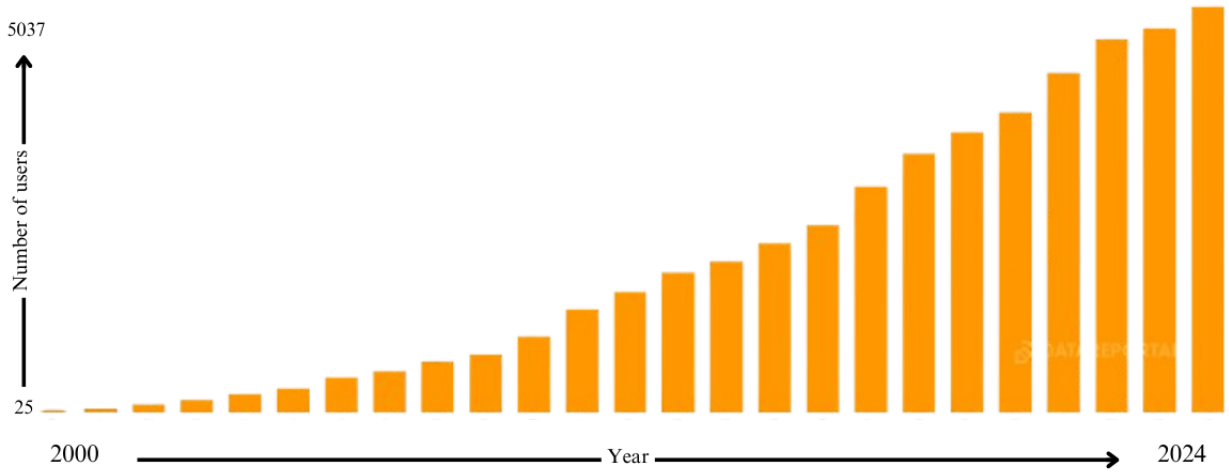

Figure 1.1: Rise of social media identities (in millions) over time
[1]

The exponential growth in users of social media over the past decade has transformed how people communicate or express their opinions. Various social media platforms have become central to global discourse, connecting billions of users worldwide. However, this common platform has brought significant challenges, particularly the rise of online hate speech and toxic comments.

Hate speech on social media encompasses a wide range of harmful expressions, including racism, sexism and other forms of discrimination and harassment. These toxic comments not only contribute to a hostile online environment but also have profound effect on mental health.

Mental Health.—Mental illnesses, such as depression, anxiety, bipolar disorder, and schizophre-

nia, are disturbingly common and can be highly debilitating. According to the Global Burden of Disease study, around a billion people in the world suffered from mental disorders in 2017, with depression and anxiety-related disorders as the leading conditions. Mental health conditions can have severe adverse effects, hampering people's ability to work, study, and be productive. [2]

1.2 Problem Statements

The latest research from GWI[3] reveals that the "typical" social media user now spends 2 hours and 23 minutes per day using social media.[1] And the more time is spent in these platforms the more toxicity is absorbed. Some media platforms have become breeding grounds for hate speech and toxic comments, significantly impacting user's mental health.

Exposure to hate speech can lead to anxiety, depression, and other severe mental health issues. For instance, the total number of individuals aged 18–23 who reported experiencing a major depressive episode increased by 83 percent between 2008 and 2018. Similarly, over the same time period, suicides became more prevalent and are now the second leading cause of death for individuals 15–24 years old. [2]

The project addresses the need for a more effective and scalable solution to identify and remove hate comments in real time with an aim to foster a more positive and inclusive online environment.[4]

1.3 Objectives

- Create a model capable of detecting hate speech with decent accuracy, considering the English language
- Implement a system that can identify and remove hate comments on social media-like platform.
- The ultimate goal is to protect users' mental health and foster a more positive and inclusive online environment

1.4 Scope

- Exploring and Analyzing available comprehensive dataset of social media comments, including both hate speech and neutral or positive comments.
- Developing a machine learning model using techniques such as deep learning (e.g., LSTM, BERT).
- Creating a simple website that simulates a social media environment.

2. Literature Review

2.1 Related Works

In recent years, various frameworks and models have been developed to detect hate speech across multiple languages and platforms. The HateClassify Framework [5] presents a scalable approach for multilingual hate speech detection on social media. It utilizes a dataset of labeled tweets and employs features such as bag-of-words, n-grams, and sentiment analysis. Machine learning models like Support Vector Machines (SVM) and Random Forests are used within this framework, demonstrating effectiveness across diverse social media platforms and languages.

Another approach involves a Multi-Class Classification System [6] designed to categorize hate speech based on severity levels. This system leverages classifiers such as Decision Trees, Naive Bayes, and SVM to differentiate between varying intensities of harmful content. Such a system is handy for identifying and prioritizing hate speech based on potential harm and enhancing content moderation efforts on social media.

An additional method explored is Hate Speech Detection with Ensemble Learning. This study applies an ensemble learning technique that combines multiple classifiers to improve hate speech detection accuracy. The approach shows promising results for cross-platform applications, as it effectively aggregates the strengths of various models to boost performance in identifying hate speech.

In[7], a Backpropagation Neural Network (BPNN) is used for hate speech detection, drawing data from Twitter and applying feature extraction methods such as Term Frequency-Inverse Document Frequency (TF-IDF) and chi-squared. The BPNN achieved an accuracy of 94.2%, illustrating its effectiveness and potential for expansion to other languages and social media platforms.

Furthermore, a study using Multinomial Logistic Regression[8] on Twitter focuses on detecting hate speech by analyzing features like sentiment, emotion, and n-grams. This model achieved an accuracy of 88.23%, highlighting its capacity to identify hate speech with notable accuracy, particularly in sentiment-rich environments like Twitter.

These studies underscore the diversity of machine learning techniques applied to hate speech detection, each with its strengths in accuracy, scalability, and adaptability across different languages and social media platforms.

2.2 Classifiers

In sentiment analysis, particularly for the classification of hate comments on social media, various classifiers are employed to categorize text into positive, negative, or neutral sentiments, or to specifically identify hate speech. Some of the algorithms are traditional machine learning approaches.

2.2.1 Naive Bayes

The Naïve Bayes algorithm is a supervised learning algorithm, which is based on the Bayes theorem and used for solving classification problems. It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.

Naive Bayes uses Bayes' Theorem to calculate the posterior probability of a class given a feature, which is expressed as:

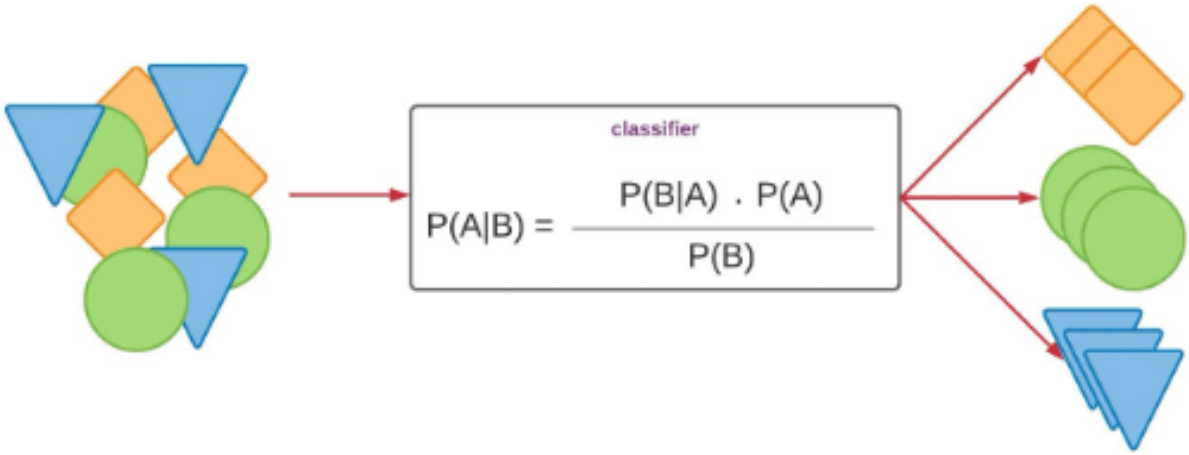

Figure 2.1: Naive Bayes Classifier

2.2.2 Decision Tree

A decision tree is a decision-support recursive partitioning structure that uses a tree-like model of decisions and their possible consequences, including chance event outcomes, resource costs, and utility. It is one way to display an algorithm that only contains conditional control statements.

A decision tree is a flowchart-like structure in which each internal node represents a "test" on an attribute (e.g. whether a coin flip comes up heads or tails), each branch represents the outcome of the test, and each leaf node represents a class label (decision taken after computing all attributes). The paths from root to leaf represent classification rules.

A decision tree consists of three types of nodes:

- Decision nodes – typically represented by squares
- Chance nodes – typically represented by circles
- End nodes – typically represented by triangles

ID3 Algorithm

A well-known decision tree approach for machine learning is the Iterative Dichotomiser 3 (ID3) algorithm. By choosing the best characteristic at each node to partition the data depending on information gain, it recursively constructs a tree. The goal is to make the final subsets as homogeneous as possible. By choosing features that offer the greatest reduction in entropy or uncertainty, ID3 iteratively grows the tree. It uses entropy and information gain as metrics.

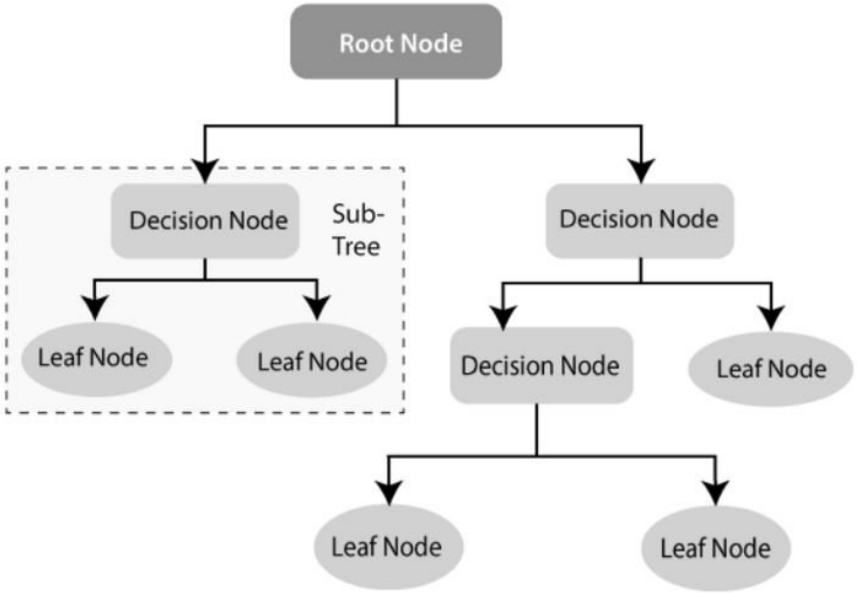

Figure 2.2: Decision Tree Structure
[9]

Entropy

Entropy $H(S)$ is a measure of the amount of uncertainty in the (data) set S (i.e., entropy characterizes the (data) set S).

$$H(S) = \sum_{c \in C} -p(c) \log_2 p(c)$$

Where,

- S – The current (data) set for which entropy is being calculated (changes every iteration of the ID3 algorithm).
- C – Set of classes in S, $C = \{\text{yes, no}\}$.
- $p(c)$ – The proportion of the number of elements in class c to the number of elements in set S.

When $H(S) = 0$, the set S is perfectly classified (i.e., all elements in S are of the same class).

In ID3, entropy is calculated for each remaining attribute. The attribute with the **smallest** entropy is used to split the set S on this iteration. The higher the entropy, the higher the potential to improve the classification here.

Information Gain

Information gain $IG(A)$ is the measure of the difference in entropy from before to after the set S is split on an attribute A. In other words, it measures how much uncertainty in S was reduced after splitting set S on attribute A.

$$IG(A, S) = H(S) - \sum_{t \in T} p(t)H(t)$$

Where,

- $H(S)$ – Entropy of set S.
- T – The subsets created from splitting set S by attribute A such that $S = \bigcup_{t \in T} t$.
- $p(t)$ – The proportion of the number of elements in t to the number of elements in set S.
- $H(t)$ – Entropy of subset t.

In ID3, information gain can be calculated (instead of entropy) for each remaining attribute. The attribute with the **largest** information gain is used to split the set S on this iteration.

2.3 Logistic Regression in Hate Speech Classification

Logistic regression is a widely used statistical method for binary classification problems. It models the probability that a given input belongs to a particular class. In the context of hate speech classification, logistic regression can effectively distinguish between hateful and non-hateful content based on features extracted from text data.

The model operates on the principle of a logistic function (also known as the sigmoid function), which maps any real-valued number into a value between 0 and 1. This is particularly useful in classification tasks, as it allows for the interpretation of the output as a probability.

The logistic regression model is represented mathematically as follows:

$$P(Y = 1|X) = \sigma(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_n X_n)}} \quad (2.1)$$

Where:

- $P(Y = 1|X)$ is the probability that the output Y is 1 (indicating the presence of hate speech) given the input features X.
- $\sigma(z)$ is the logistic function.
- β_0 is the intercept of the model.
- $\beta_1, \beta_2, \ldots, \beta_n$ are the coefficients of the features $X_1, X_2, \ldots, X_n$, which represent the weights assigned to each feature.

Logistic regression is advantageous due to its simplicity, interpretability, and efficiency, making it suitable for initial exploratory analyses and baseline models in hate speech classification tasks.

2.4 BERT

It is a Bidirectional Encoder Representation from Transformers. BERT is a state-of-the-art transformer-based model that has revolutionized natural language processing (NLP), including sentiment analysis tasks. Unlike traditional models that read text sequentially (left-to-right or right-to-left), BERT reads the entire sequence of words at once, capturing both left and right context in all layers. This bidirectional approach allows BERT to understand the nuanced meaning of words in context. BERT's ability to capture context and its pre-trained knowledge make it exceptionally powerful for sentiment analysis, especially in understanding complex and ambiguous language used in social media.

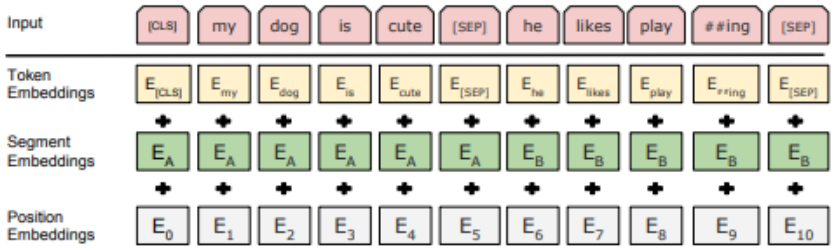

Figure 2.3: BERT input representation
[10]

2.5 Word Cloud Visualization

Word Cloud Visualization is a popular technique used to represent text data visually. It displays the most frequent words in a text corpus, with the size of each word indicating its frequency or importance. Word cloud visualization is a powerful tool for quickly summarizing and presenting the key elements of a text corpus. It is particularly useful in exploratory data analysis, where the goal is to gain an initial understanding of the data's content. In sentiment analysis, word clouds can help highlight the most common terms associated with different sentiments, providing valuable insights into the language used in social media, reviews, or other text-based data.

3. Methodology

3.1 Requirement Analysis

The Requirement Analysis aims to define the functional and non-functional requirements of the project to ensure its successful implementation. This section outlines the core functionalities and expectations from the application.

3.1.1 Functional Requirements

The application must fulfill the following functional requirements:

Loading Comments

The application should be capable of retrieving comments from the backend database and displaying them in the user interface efficiently and in real-time.

Comment Classification

The application should classify both previously stored comments and new comments posted by users. The classification should categorize comments based on their content (e.g., toxic or non-toxic).

Storing Classification Results

The predictions made by the application should be securely stored in the backend database. This ensures that the classifications can be accessed and analyzed in the future if needed.

Hiding Toxic Comments

The application should provide an option to hide toxic comments from the interface. When the user presses the Hide button, the toxic comment should be dynamically removed from view.

3.1.2 Non-Functional Requirements

The non-functional requirements define the quality attributes and constraints of the application, ensuring it meets user expectations and operates effectively under various conditions.

Performance

The model should achieve an accuracy rate of 75

Usability

The application should feature a user-friendly interface, allowing end-users to navigate and interact with its features seamlessly. The design should prioritize simplicity, intuitive operation, and smooth functionality to enhance the user experience.

Scalability

The application must efficiently handle larger datasets without significant performance degradation. It should be robust enough to scale up as the volume of comments or user activity increases.

3.2 Dataset Collection

In this project, we have used three different datasets to detect and classify hate speech, toxic comments, and harmful language across different online platforms. The datasets include text data from Twitter, Wikipedia comments, and YouTube, each with specific characteristics and annotations. They are discussed in brief in this section:

3.2.1 Hate Speech and Offensive Language Dataset

It is a collection of tweets labeled for hate speech, offensive language, or neither. This dataset captures the unique characteristics of short-form social media text, making it ideal for analyzing hate speech patterns on platforms where informal language, abbreviations, and unique syntax are common.
Source: Twitter
Data Composition: It consists of **25.3k+** unique tweets. Each tweet is annotated with labels indicating "Hate Speech," "Offensive Language," or "Neither."
Purpose: Used to identify and classify hate speech within Twitter's short-text format, aiding in understanding hate speech manifestations in informal online discourse.

3.2.2 Google's Jigsaw Toxic Comment Dataset

Google created the Jigsaw Toxic Comment Dataset[11] which contains comments from Wikipedia labeled across multiple toxicity-related categories. The dataset includes various degrees of harmful language, allowing for a more comprehensive analysis of toxic behavior online.
Source: Wikipedia comments

Data Composition: It consists of **159.5k+** unique comments. Each comment is labeled with categories such as "Toxic," "Severe Toxic," "Obscene," "Threat," "Insult," and "Identity Hate."

Purpose: Provides a rich, multi-label dataset for understanding a range of toxic behaviors, ideal for developing nuanced classification models that recognize multiple forms of online toxicity.

3.2.3 YouToxic Dataset

The YouToxic Dataset consists of a collection of English-language YouTube comments specifically annotated for toxicity. This dataset helps capture toxic behavior in video comment sections, where discussions may be longer and contextually different from platforms like Twitter or Wikipedia.

Source: YouTube comments

Data Composition: **1,000** comments labeled with categories such as "IsToxic," "IsAbusive," "IsThreat," "IsProvocative," "IsObscene," "IsHatespeech," "IsRacist," "IsNationalist," "IsSexist," "IsHomophobic," "IsReligiousHate," and "IsRadicalism."

Purpose: Supports analysis of toxic language in YouTube's video-centric platform, adding another layer of diversity to the toxic content detection task.

3.2.4 SemEval-2018 Task 1: Affect in Tweets

The **SemEval 2018 Task 1 Dataset** [12] is a collection of English-language tweets specifically annotated for various emotions. This dataset enables the analysis of emotional states expressed in text, providing a nuanced understanding of

Source: Tweets from Twitter.

Data Composition: Includes thousands of tweets labeled with emotions such as *anger*, *anticipation*, *disgust*, *fear*, *joy*, *love*, *optimism*, *pessimism*, *sadness*, *surprise*, and *trust*. Each tweet is annotated to indicate whether it expresses one or more of these emotions.

Purpose: Designed to support the study of emotional language on social media, this dataset is valuable for tasks such as emotion recognition, sentiment analysis, and psychological behavior modeling, with applications in mental health analysis, marketing, and human-computer interaction.

3.3 Data Exploration

Data exploration is a crucial step in understanding the dataset, identifying patterns, and preparing it for further processing and model training. This phase provides a comprehensive overview of the dataset's structure, label distribution, and key text characteristics.

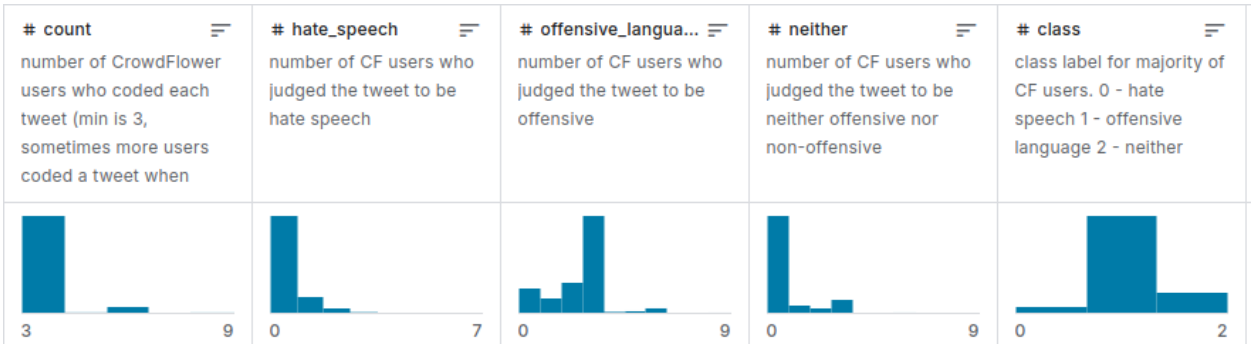

Figure 3.1: Hate Speech and Offensive Language Dataset description [13]

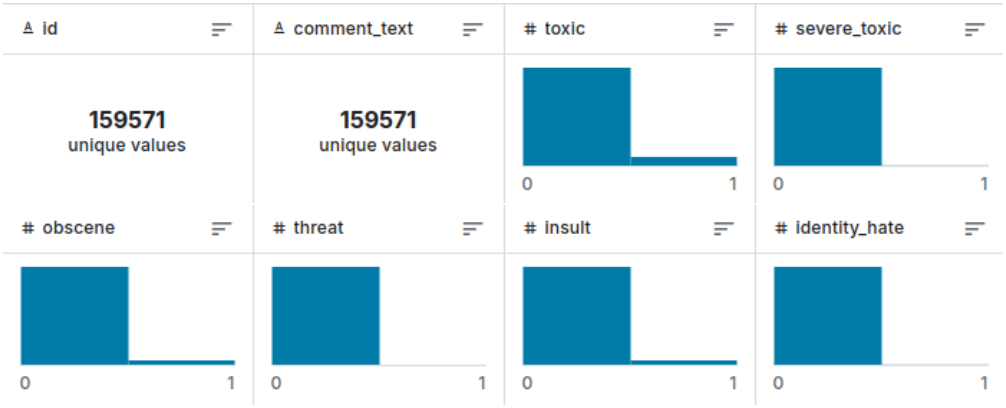

Figure 3.2: Google's Jigsaw Toxic Comments Dataset Description

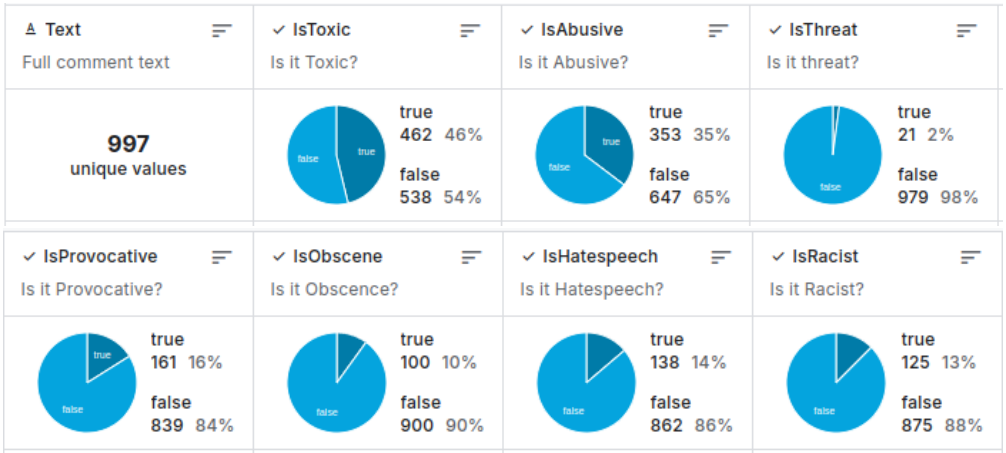

Figure 3.3: Youtube Toxic Comments Dataset Description [14]

	train	validation	test
English	6,838	886	3,259

Figure 3.4: SemEval-2018 Data Split

Figure 3.5: Word Cloud visualization of Hate Speech and Offensive Language Dataset description

Figure 3.6: Word Cloud visualization of Jigsaw Toxic Dataset [Train.csv]

Figure 3.7: Word Cloud visualization of Youtube Toxic Comments Dataset Description

3.4 Data Preprocessing

The raw text data was passed through several preprocessing steps to standardize and prepare it for further analysis. The combined preprocessing pipeline ensures the text data is clean, consistent, and suitable for downstream NLP tasks. These steps include:

3.4.1 Stopword Removal

Stopwords, which are common words like *"the,"* *"is,"* and *"and,"* that contribute little to the semantic meaning, were removed. This step reduces noise and focuses on the most informative terms in the text.

3.4.2 Text Cleaning

The text was converted to lowercase to ensure uniformity. Various contractions (e.g., *"what's"* to *"what is"*) were expanded to their full forms. Non-alphanumeric characters were removed, and excessive whitespace was eliminated to improve text quality.

3.4.3 Stemming

Stemming was applied using the Snowball Stemmer to reduce words to their root forms. For example, words like *"running"* and *"runs"* were normalized to *"run."* This step reduces vocabulary size while preserving meaning.

These were the preprocessing steps we applied for training the classical methods on the mentioned datasets.

3.4.4 BERT Tokenization

For fine-tuning with BERT, the `AutoTokenizer` from the Hugging Face Transformers library was used. Specifically, the `bert-base-cased` pre-trained tokenizer was employed. The tokenizer performs the following key operations:

- **WordPiece Tokenization:** Text is split into subword units, allowing the model to handle rare and out-of-vocabulary words effectively.
- **Adding Special Tokens:** Special tokens, such as `[CLS]` and `[SEP]`, were added to indicate the start and separation of input sequences.

Using the `AutoTokenizer.from_pretrained("bert-base-cased")`, the text data was tokenized and converted into numerical input IDs, ready for training with the BERT model.

4. Model Selection and Training

4.1 Classical Machine Learning Approaches

We experimented with several classical machine learning models to classify toxicity in comments across the previously discussed datasets: the Twitter Hate Speech Dataset, Google's Jigsaw Toxic Comment Dataset, and the YouToxic Dataset. The following models were utilized:

4.1.1 Logistic Regression

A logistic regression model predicts a dependent data variable by analyzing the relationship between one or more existing independent variables [8]. The model's ability to handle binary and multilabel classification tasks made it a suitable choice for distinguishing between various forms of toxicity present in the comments and posts. By analyzing features derived from the text using Term Frequency-Inverse Document Frequency (TF-IDF), models were trained on the three datasets. The results are shown below.

- **Hate Speech and Offensive Language Dataset (Twitter)**
  Logistic Regression Accuracy: 0.8737
  Logistic Regression Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Hate Speech Detected	0.37	0.54	0.44	290
Offensive Language Detected	0.96	0.89	0.92	3832
No hate or offensive speech	0.77	0.91	0.84	835
Accuracy	0.8737			

Table 4.1: Logistic Regression Classification Report - Twitter Dataset

- **Google's Jigsaw Toxic Comment Dataset (Wikipedia)**
  Logistic Regression Accuracy: 0.9187
  Logistic Regression Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Toxic	0.91	0.62	0.74	3056
Severe Toxic	0.58	0.28	0.38	321
Obscene	0.92	0.64	0.75	1715
Threat	0.69	0.15	0.24	74
Insult	0.81	0.51	0.62	1614
Identity Hate	0.72	0.16	0.26	294
Accuracy	0.9187			

Table 4.2: Logistic Regression Classification Report - Wikipedia Dataset

- **YouToxic Dataset (Youtube)**
  Logistic Regression Accuracy: 0.66
  Logistic Regression Classification Report

Class	Precision	Recall	F1-score	Support
0	0.58	0.45	0.51	55
1	0.69	0.57	0.62	102
2	0.66	0.80	0.72	143
Accuracy	0.66 (300)			

Table 4.3: Logistic Regression Classification Report - Youtube Toxic

4.1.2 Naive Bayes

It is a probabilistic classifier based on applying Bayes' theorem with strong independence assumptions.

- **Hate Speech and Offensive Language Dataset (Twitter)**
  Naive Bayes Accuracy: 0.7821
  Naive Bayes Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Hate Speech Detected	0.00	0.00	0.00	290
Offensive Language Detected	0.78	1.00	0.88	3832
No hate or offensive speech	0.98	0.05	0.10	835
Accuracy	0.7821			

Table 4.4: Naive Bayes Classification Report - Twitter Dataset

- **Google's Jigsaw Toxic Comment Dataset (Wikipedia)**

  Naive Bayes Accuracy: 0.8998 Naive Bayes Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Toxic	0.99	0.19	0.32	3056
Severe Toxic	0.00	0.00	0.00	321
Obscene	0.98	0.11	0.20	1715
Threat	1.00	0.01	0.03	74
Insult	0.97	0.05	0.10	1614
Identity Hate	0.00	0.00	0.00	294
Accuracy	0.8998			

Table 4.5: Naive Bayes Classification Report - Wikipedia Dataset

- **YouToxic Dataset (Youtube)**

  Naive Bayes Accuracy: 0.51

  Naive Bayes Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Hateful	0.00	0.00	0.00	55
Non-hateful	0.49	0.99	0.66	143
Offensive	0.83	0.10	0.18	102
Accuracy	0.51			

Table 4.6: Naive Bayes Classification Report - Youtube Dataset

4.1.3 Decision Tree

It is a model that uses a tree-like graph to make decisions based on feature values.

- **Hate Speech and Offensive Language Dataset (Twitter)**

  Decision Tree Accuracy: 0.8628

  Decision Tree Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Hate Speech Detected	0.34	0.33	0.34	465
No hate or offensive speech	0.82	0.74	0.78	1379
Offensive Language Detected	0.91	0.93	0.92	6335
Accuracy	0.8628			

Table 4.7: Decision Tree Classification Report - Twitter Dataset

- **G**oogle's Jigsaw Toxic Comment Dataset (Wikipedia)
  Decision Tree Accuracy: 0.8921
  Decision Tree Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Toxic	0.70	0.70	0.70	3056
Severe Toxic	0.32	0.24	0.27	321
Obscene	0.77	0.75	0.76	1715
Threat	0.23	0.23	0.23	74
Insult	0.64	0.60	0.62	1614
Identity Hate	0.43	0.34	0.38	294
Accuracy	0.8921			

Table 4.8: Decision Tree Classification Report - Wikipedia Dataset

- **YouToxic Dataset (Youtube)**
  Decision Tree Accuracy: 0.62
  Decision Tree Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Hateful	0.59	0.18	0.28	55
Non-hateful	0.64	0.83	0.72	143
Offensive	0.58	0.55	0.56	102
Accuracy	0.62			

Table 4.9: Decision Tree Classification Report - Youtube

4.2 Fine-Tuning BERT for Sequence Classification

As we discussed above in the Literature Review section, BERT stands for Bidirectional Encoder Representations from Transformers. The name itself is self-explanatory for what BERT is all about.

BERT architecture consists of several Transformer encoders stacked together. Each Transformer encoder encapsulates two sub-layers: a self-attention layer and a feed-forward layer.

4.2.1 BERT models:

There are two different BERT models:

- BERT base: It consists of 12 layers of Transformer encoder, 12 attention heads, 768 hidden size, and 110M parameters.
- BERT large: It consists of 24 layers of Transformer encoder, 16 attention heads, 1024 hidden size, and 340 parameters.

BERT Input and Output models expect a sequence of tokens(words) as an input. In each sequence of tokens, there are two special tokens the BERT would expect as input:

- [CLS]: This is the first token of every sequence, which stands for classification token.
- [SEP]: This is the token that lets BERT know which token belongs to with sequence. This special token is mainly important for a next-sentence prediction task or question-answering task. If we only have one sequence, then this token will be appended to the end of the sequence.

It is also important to note that the maximum size of tokens that can be fed into the BERT model is 512. If the tokens in a sequence are less than 512, we can use padding to fill the unused token slots with the [PAD] token. If the tokens in a sequence are longer than 512, then we need to do a truncation.

Bert model then will output an embedding vector of size 768 in each of the tokens. We can use these vectors as input for different kinds of NLP applications, whether it be text classification, next-sentence prediction, question-answering, or Named-Entity-Recognition (NER).

4.2.2 RoBERTa: A Robustly Optimized BERT Pretraining Approach

RoBERTa, introduced in "RoBERTa: A Robustly Optimized BERT Pretraining Approach,"[15] refines BERT's pretraining methodology by leveraging larger datasets, longer training times, and dynamic masking to enhance performance on downstream tasks. Unlike BERT, RoBERTa removes the Next Sentence Prediction (NSP) objective, focuses on training with larger batch size, and utilizes more robust hyperparameters. These modifications make RoBERTa more effective for fine-tuning specific tasks, outperforming BERT on several benchmarks.

4.2.3 Text Classification:

In this text classification task, we use the BERT model by focusing on the embedding vector created by the special [CLS] token. The [CLS] token is added at the beginning of each input text and is meant to represent the overall meaning of the entire text.[16]

During training (fine-tuning) for hate speech and offensive language detection, BERT processes the text, and the [CLS] token's final embedding (a 768-dimensional vector) captures the essence of the input text. This vector is then passed into a classifier layer, which produces an output prediction across three categories: Hate Speech, Offensive Language. or Neither.[17]

Using this approach, the model can classify complex language patterns effectively, without removing the nuances in the text as BERT can understand the context either way.

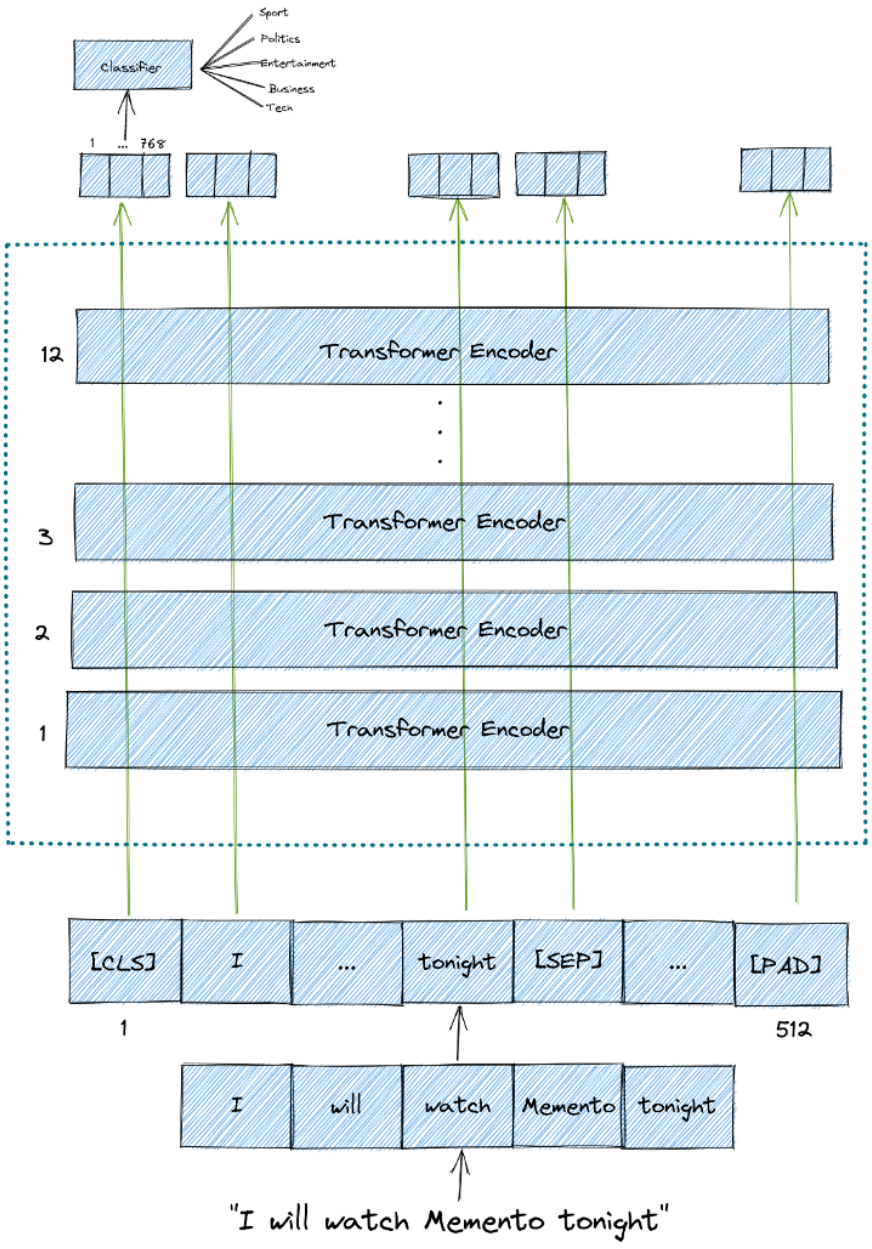

Figure 4.1: Fine-Tuning a BERT model for sequence classification [18]

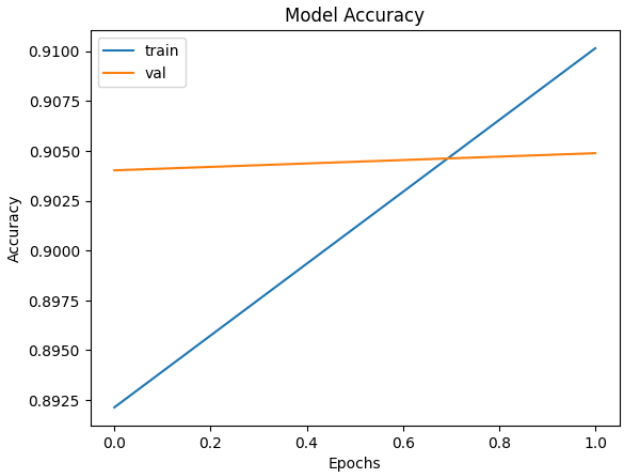

Figure 4.2: Model Accuracy at 2 epochs trained on Twitter dataset

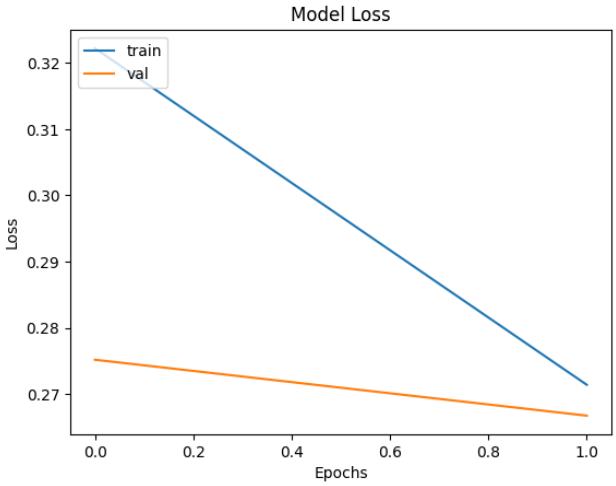

Figure 4.3: Model Loss at 2 epochs trained on Twitter dataset

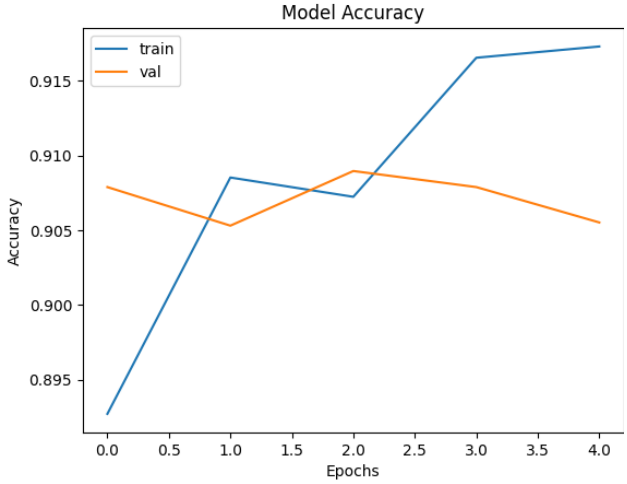

Figure 4.4: Model Accuracy at 5 epochs trained on Twitter dataset

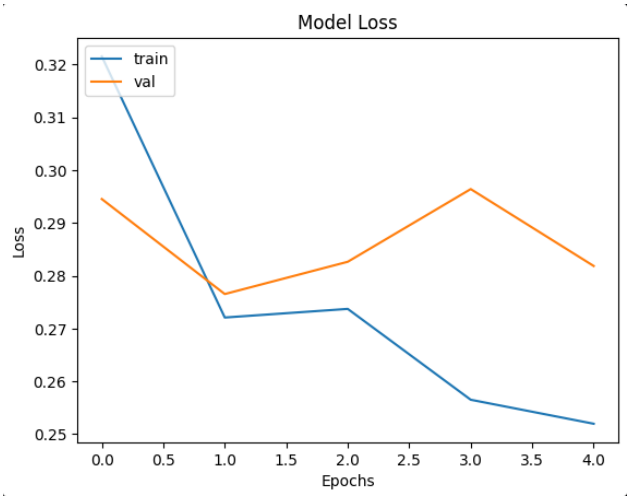

Figure 4.5: Model Loss at 5 epochs trained on Twitter dataset

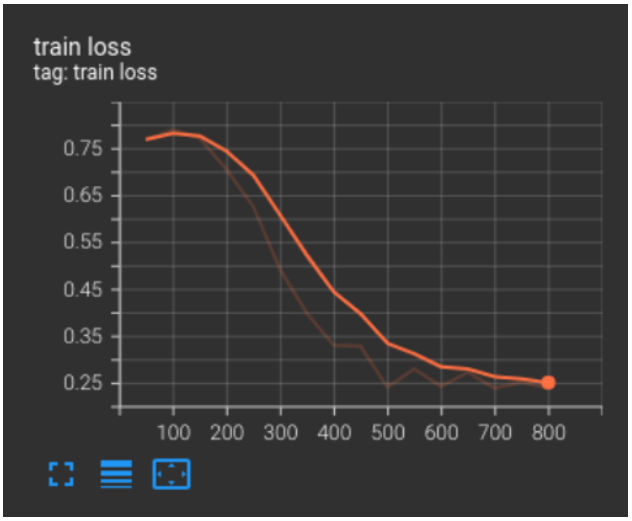

Figure 4.6: Training Loss when fine-tuning BERT for Google Jigsaw Toxic Comments Dataset

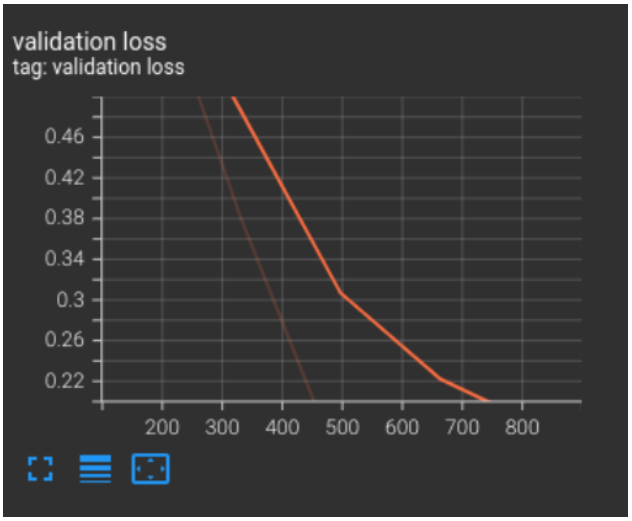

Figure 4.7: Validation Loss when fine-tuning BERT for Google Jigsaw Toxic Comments Dataset

4.3 Evaluation Metrics

4.3.1 Precision

Precision is the ratio of true positive predictions to the total predicted positives. It measures how many of the predicted "hate speech" instances were actually correct. High precision ensures that when the model labels content as hate speech, it's rarely wrong.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

4.3.2 Recall

Recall (or Sensitivity) is the ratio of true positive predictions to the total actual positives. It measures how many of the actual hate speech instances the model correctly identified. High recall ensures that most hate speech content is detected by the model.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

4.3.3 F1-score

F1-score is the harmonic mean of precision and recall, providing a balance between the two metrics. It is useful when we need to balance the trade-off between precision and recall.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

4.3.4 Auc-Roc Curve

The Area Under the Receiver Operating Characteristic Curve (AUC-ROC) represents the ability of the model to distinguish between classes. The ROC curve plots the True Positive Rate (Recall) against the False Positive Rate (1 - Specificity) at various threshold levels.

For hate speech classification, where both false positives and false negatives are critical, F1-score provides a single metric to evaluate the model comprehensively.

5. Result and Discussion

The results of the **Sweep Toxic Comments Classification Project** demonstrate the effectiveness of both classical and modern machine learning approaches in detecting toxic comments across multiple datasets, including Twitter, Wikipedia, and YouTube comments.

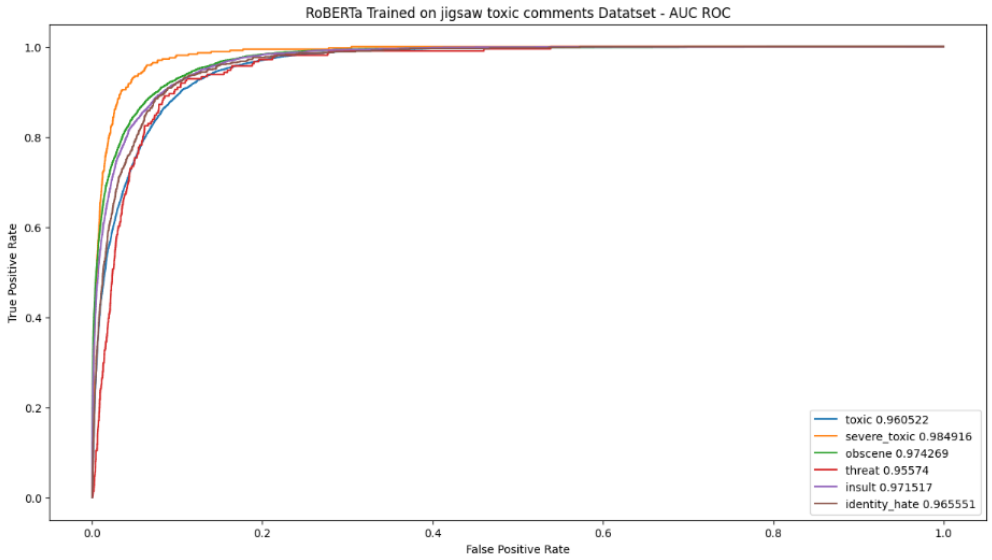

Figure 5.1: RoBERTa Tested on jigsaw toxic comments Datatset - AUC ROC

Epoch	Training Loss	Validation Loss	F1	Roc Auc	Accuracy
1	0.407100	0.313707	0.687967	0.783506	0.281038
2	0.281700	0.301289	0.698197	0.787363	0.283296
3	0.240100	0.305926	0.714180	0.805066	0.292325
4	0.211600	0.311311	0.700770	0.796476	0.266366
5	0.191600	0.310656	0.706203	0.799846	0.277652

Figure 5.2: Fine-tuned BERT for multi-label text classification

5.1 Results Across Models

5.1.1 Classical Machine Learning Approaches

- **Logistic Regression**: Achieved an accuracy ranging from **66% to 91.8%** across datasets. It excelled in binary classification tasks but struggled with nuanced multi-label classifications, such as detecting severe toxic or identity hate categories.

```
test_data = "you stupid"
test_data_transformed = tfidf.transform([test_data])
predicted_class = logistic_model.predict(test_data_transformed)
```

```
predicted_label = label_mapping[predicted_class[0]]

print(f"Predicted Class: {predicted_class[0]}")
print(f"Predicted Label: {predicted_label}")
```

```
Predicted Class: 0
Predicted Label: Hate Speech Detected
```

- **Naive Bayes**: Showed lower performance (accuracy between **51% and 89.9%**), largely due to its assumption of feature independence, which does not hold in complex textual data.

```
test_data = "you are a good"
test_data_tfidf = tfidf_nb.transform([test_data])
predicted_class = nb_model.predict(test_data_tfidf)
predicted_label = label_mapping[predicted_class[0]]
print(f"\nPredicted Label for new data: {predicted_label}")
```

```
Predicted Label for new data: Offensive Language Detected
```

Figure 5.3: Output in Hate Speech and Offensive Language Dataset

- **Decision Tree**: Provided a balanced approach, achieving accuracies between **62% and 89.2%**, but suffered from overfitting, particularly in smaller datasets like YouToxic.

```
test_data = "you are a good"
df = cv_dt.transform([test_data]).toarray()
print(dt_model.predict(df))
```

```
['No hate or offensive speech']
```

Figure 5.4: Output in Hate Speech and Offensive Language Dataset

5.1.2 Fine-Tuned BERT

The BERT-based model outperformed classical methods, achieving higher AUC scores and demonstrating superior ability to capture contextual nuances in text. The inclusion of pre-

trained embeddings allowed BERT to generalize better, particularly for edge cases and multi-label categories, such as "Severe Toxic" and "Threat."

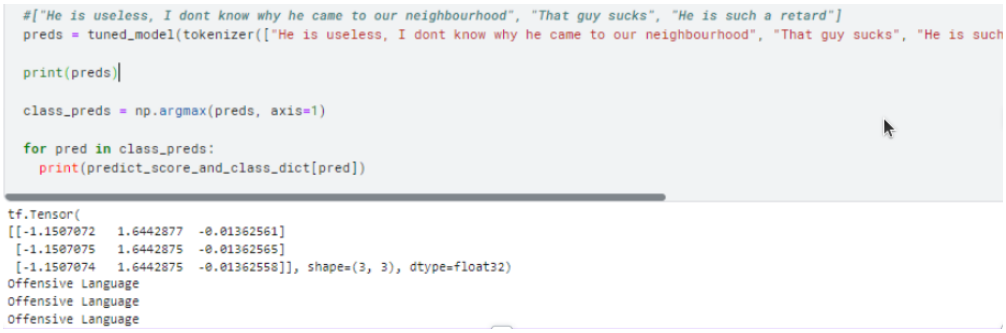

Figure 5.5: Output of fine tuned BERT in Hate Speech and Offensive Language Dataset

5.2 Dataset-Specific Observations

- **Twitter Hate Speech Dataset**: The models faced challenges with informal language, abbreviations, and unique syntax. BERT handled these better due to subword tokenization but still showed room for improvement in classifying nuanced categories.
- **Wikipedia Toxic Comment Dataset (Jigsaw)**: With its richer annotation and larger size, this dataset enabled all models to perform better. However, categories like "Threat" and "Identity Hate" remained difficult, with low recall values even for BERT.
- **YouToxic Dataset**: The smaller dataset size posed challenges for all models.

5.3 Discussion

5.3.1 Performance Analysis

BERT emerged as the best-performing model, showcasing the value of transformer architectures for text classification tasks. However, its computational requirements were significantly higher than classical models. Logistic Regression offered a fast and interpretable baseline but was limited by its reliance on linear decision boundaries. Naive Bayes, while simple and computationally efficient, struggled to capture the complexities of toxic comment classification.

5.3.2 Error Patterns

Across all datasets, the models struggled with comments containing ambiguous or sarcastic language. Misclassifications often occurred in categories with limited training data, such as "Threat" or "Identity Hate."

The results underscore the importance of contextual understanding in toxic comment classification. While BERT's performance is promising, the lower recall in sensitive categories like "Threat" suggests potential risks in real-world deployment, where missing such cases could have serious consequences.

6. Web Application Development

To demonstrate the functionality of our system, we developed a web application that showcases how the system operates. The web application development process was divided into two main parts: the Frontend and the Backend.

6.1 Frontend

For the frontend development, we utilized the React framework for JavaScript, complemented by CSS for styling. The design aimed to resemble the newsfeed of a typical social media application, providing an intuitive and familiar user experience.

Key features of the frontend include:

- Post and Comment Interaction: Users can post comments under a post, which are then displayed beside the post in a thread-like structure.
- Hide Button: Each comment has a "Hide" button that allows users to filter and hide toxic comments. This feature dynamically updates the interface to remove such comments from view.

The frontend was designed to ensure a smooth and interactive user experience, reflecting the practical usage of the classification system.

6.2 Backend

For the backend, we employed a combination of Express.js and FastAPI to handle API calls and integrate the classification model effectively.

Express.js:

Used for managing API endpoints and ensuring communication between the frontend and backend. Two API endpoints were created:

- GET Endpoint: Fetches and returns comments to display on the user interface.
- POST Endpoint: Accepts new comments, classifies them, and stores the results for future use.

FastAPI:

Used to load the classification model and handle comment classification. An endpoint was created to process incoming comments, classify them as toxic or non-toxic, and return the results to the Express.js API. The backend architecture effectively integrates the model with the web application, ensuring seamless functionality. The system not only processes and classifies user comments but also stores the results securely for future reference.

The web application was developed to resemble a social media newsfeed page, serving as a demonstration of the system's core functionality. While it is not a full-fledged product, the application highlights the system's features, including comment classification, toxic comment filtering, and user interaction capabilities. This prototype successfully showcases the potential applications of the classification model in a real-world scenario.

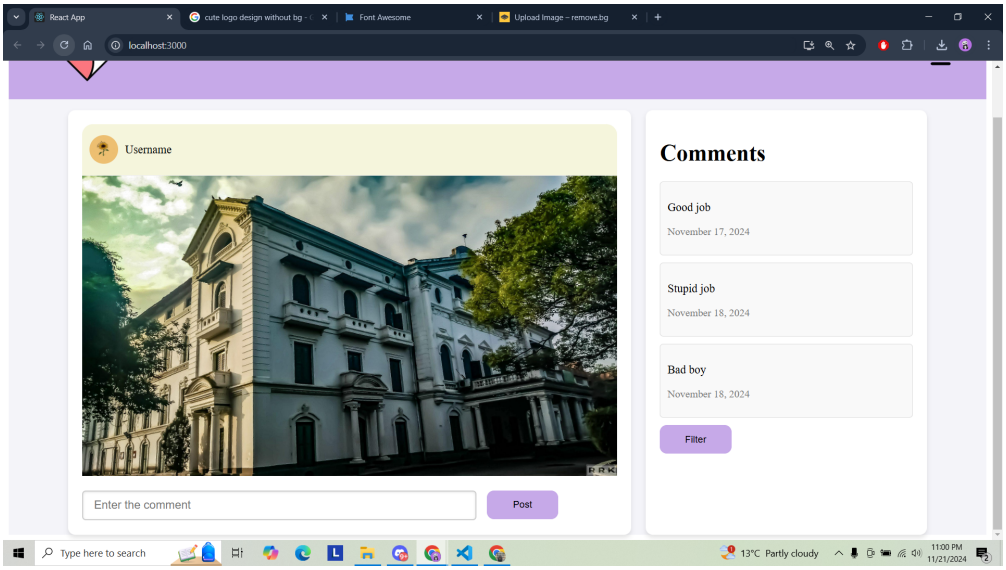

Figure 6.1: Web Application Interface

7. Conclusions

The Sweep Toxic Comments Classification Project created a system to identify and classify toxic comments using both traditional machine learning approaches and fine tuning advanced deep learning methods like BERT. The results showed that fine-tuned models worked better compared to the traditional machine learning techniques, achieving good accuracy and f1-score, and demonstrated the system's ability to help reduce hate speech and promote a positive online environment.

The project also included a prototype web application with a simple and easy-to-use interface. It demonstrated how the system could function in real-life situations. While there are still issues with accuracy and scalability, the project lays a solid foundation for future improvements, such as boosting model performance and integrating the system as a plugin on actual social media platforms.

8. Limitation and Future Enhancement

8.0.1 Limitations

While the developed web application successfully demonstrates the functionality of the classification system, several limitations were identified:

Basic Frontend Design

The frontend design, while functional, is simplistic and lacks advanced features such as user authentication, multi-post support, or dynamic loading of comments.

Limited Scalability

The current backend implementation may struggle to handle high volumes of user activity or comments due to limited optimizations for large-scale data processing.

Accuracy of Classification

The model achieves 75–80 percentage accuracy, which may not be sufficient for highly sensitive applications. Misclassifications of toxic comments could lead to user dissatisfaction.

Storage Mechanism

The backend stores comments and classification results in a file, which is not scalable for larger datasets. A robust database system is not yet integrated.

8.0.2 Future Enhancements

To overcome the above limitations and expand the application's potential, the following enhancements are proposed:

Scalable Backend Infrastructure

- Integrate a database system, such as MongoDB or PostgreSQL, for scalable storage of comments and classification results.
- Optimize the backend to handle a higher volume of API calls and concurrent users.

Improved Classification Model

- Fine-tune the classification model further to improve accuracy and handle edge cases.
- Integrate ensemble techniques or additional layers to boost performance.

Multi-Language Support

Extend the application to support classification of comments in multiple languages to cater to a broader audience.

Plugins and Extensions

The development of plugins and extensions would enable the classification system to be integrated into various platforms, such as social media sites, blogs, or forums. These tools would extend the functionality of the application, allowing seamless classification of comments on existing platforms without requiring users to leave the platform.

By addressing these limitations and implementing the suggested enhancements, the web application can evolve into a fully functional, scalable, and user-friendly system suitable for real-world applications.

References

[1] Datareportal. Digital 2024 global overview report, 2024. Accessed: 2024-08-11.

[2] Luca Braghieri, Ro'ee Levy, and Alexey Makarin. Social media and mental health. *American Economic Review*, 112(11):3660–3693, 2022.

[3] GWI. Gwi - audience targeting and consumer insights platform. https://www.gwi.com/, 2024. Accessed: 2024-08-11.

[4] Greater Good Science Center. Can we end hate speech on social media? https://greatergood.berkeley.edu/article/item/can$_w e_e nd_h ate_s peech_o n_s ocial_m edia$, 2020. *Accessed* : $2024 - 10 - 30$.

[5] M. U. S. Khan, A. Abbas, A. Rehman, and R. Nawaz. HateClassify: A Service Framework for Hate Speech Identification on Social Media. *IEEE Internet Computing*, 25(1):40–49, Jan.-Feb. 2021.

[6] K. A. Qureshi and M. Sabih. Un-Compromised Credibility: Social Media Based Multi-Class Hate Speech Classification for Text. *IEEE Access*, 9:109465–109477, 2021.

[7] N. A. Setyadi, M. Nasrun, and C. Setianingsih. Text Analysis For Hate Speech Detection Using Backpropagation Neural Network. In *2018 International Conference on Control, Electronics, Renewable Energy and Communications (ICCEREC)*, pages 159–165, 2018.

[8] Kinza Yasar. Logistic regression, n.d. Accessed: 2024-11-05.

[9] An example of a decision tree structure. https://www.researchgate.net/figure/An-example-of-a-decision-tree-structure$_f ig4_3$50297716.

[10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.

[11] Jigsaw toxic comment classification challenge, 2018.

[12] SemEval Workshop. Semeval-2018 task 1: Affect in tweets dataset, 2018.

[13] Hate speech and offensive language dataset, 2017.

[14] Reihan Namdari. Youtube toxicity data. https://www.kaggle.com/datasets/reihanenamdari/youtube-toxicity-data?select=youtoxic$_e nglish_1$000.csv, 2020. [$Dataset$].

[15] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[16] Chris J. Kennedy, Geoff Bacon, Alexander Sahn, and Claudia von Vacano. Constructing interval variables via faceted rasch measurement and multitask deep learning: a hate speech application. *arXiv preprint arXiv:2009.10277*, 2020. Accessed: 2024-10-21.

[17] Joni Salminen, Maximilian Hopf, Shammur A. Chowdhury, Soon gyo Jung, Hind Almerekhi, and Bernard J. Jansen. Developing an online hate classifier for multiple social media platforms. *Human-centric Computing and Information Sciences*, 10(1), 2020.

[18] Reference image for report, 2024.